

Question 11.1 — Command History

Question:

To store commands, maintain a history buffer, handle capacity limits, retrieve entries, display history, and validate consistency.

Question 11.2 — Pipeline Structure

Question:

To create pipeline structures, store commands, maintain execution order, connect processes, validate pipeline layout, and test configurations.

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano skill11.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skill11.c -o skill11
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./skill11
shell> ls
shell> pwd
shell> help
shell> history
1  ls
2  pwd
3  help
shell> cat skill11.c
shell> ^C
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill11.c
#include <stdio.h>
#include <string.h>

#define HISTORY_SIZE 10
#define COMMAND_SIZE 200

int main()
{
    char history[HISTORY_SIZE][COMMAND_SIZE];
    int count = 0;
    char command[COMMAND_SIZE];

    while (1)
    {
        printf("shell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';
    }
}
```

```

    if (strcmp(command, "exit") == 0)
        break;

    if (strcmp(command, "history") == 0)
    {
        for (int i = 0; i < count; i++)
        {
            printf("%d  %s\n", i + 1, history[i]);
        }

        continue;
    }

    if (count < HISTORY_SIZE)
    {
        strcpy(history[count], command);
        count++;
    }
    else
    {
        for (int i = 1; i < HISTORY_SIZE; i++)
        {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[HISTORY_SIZE - 1], command);
    }
}

return 0;
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano skilll1a.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skilll1a.c -o skilll1a
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./skilll1a

```

```
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano skill11a.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc skill11a.c -o skill11a
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./skill11a
52
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat skill11a.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == -1)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);
    }
}
```



saikrishna_90156@DESKTOP- X



```
        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 == -1)
    {
        perror("fork");
        return 1;
    }

    if (p2 == 0)
    {
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);
        close(fd[1]);

        char *args[] = {"wc", "-l", NULL};

        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
```